

Agency Blog SaaS — Demo

Deep Dive: everything in this repository, explained

What this document is about. This PDF documents the *demo* codebase at `agency-blog-saas-demo/` — a scripted, fabricated-data product tour built by the repository owner. It does *not* document `agency-blog-saas`, the real, private, production Django SaaS this demo visually and functionally imitates. Every claim below is either read directly from this demo’s own source files and README, or explicitly marked as an inference.

Contents

1	What this project is, in one paragraph	2
1.1	Why a demo exists at all	2
2	Core concepts a beginner needs first	2
3	The one honesty rule this whole repository is built around	3
4	Architecture: how the four files fit together	3
5	File-by-file walkthrough	4
5.1	<code>index.html</code> — page structure	4
5.2	<code>demo-data.js</code> — the data layer	4
5.3	<code>app.js</code> — behavior	4
5.4	<code>styles.css</code> — the <code>davonex.com</code> visual language	5
6	User flows	6
6.1	The AI Blog Generator flow	6
6.2	Reading an article	6
6.3	Tab state machine	6
7	Data model detail	7
8	How this relates to the real, private project	7
9	Challenges the owner records in the README	7
10	Setup, deployment, and licensing	8
11	What the owner says they’d change	8

1 What this project is, in one paragraph

`agency-blog-saas-demo` is a single static web page — plain HTML, CSS, and vanilla JavaScript, no server, no build tool, no framework — that lets a visitor click through the same four core screens the real, private `agency-blog-saas` product has: an AI Blog Generator, a Blog Library, Subscription Packages, and Client Updates. Nothing on the page ever talks to a network. Every interaction — typing in the generator form, clicking “Generate,” watching a progress spinner, browsing the library, reading an article — is driven entirely by JavaScript running in the visitor’s own browser against data that was written into the page ahead of time.

Static page. A web page whose HTML, CSS, and JavaScript files are fixed ahead of time and simply sent to the browser as-is, with no server-side program generating the page per request. Opening `index.html` directly, or serving the folder with any generic file server, is enough to run it.

1.1 Why a demo exists at all

The real system, `agency-blog-saas`, is a production Django SaaS running behind `digitalise.agency`: it does actual AI blog generation, manages real credits and subscriptions, has per-client editor roles, and exports PDFs, for real paying agency clients. Because its source code *is* the agency’s live commercial product — not just data that could be scrubbed of secrets and shared — it has to stay private. A hiring panel or interviewer, though, cannot form an opinion about something they can’t see or click. This demo exists to close that gap: it reproduces the real product’s actual feature set and screen structure, populated with a mix of (a) genuinely real, already-public content from one of the agency’s real clients, and (b) clearly fabricated numbers and interactions, so a visitor gets a faithful, honest impression of what the real system does without any of its private code, business logic, or other clients’ data ever leaving the private repository.

2 Core concepts a beginner needs first

HTML, CSS, and JavaScript. The three languages every web page is built from. HTML (`index.html`) describes the structure and content of the page — headings, buttons, text boxes. CSS (`styles.css`) describes how that structure looks — colors, spacing, fonts, layout. JavaScript (`app.js`, `demo-data.js`) is a real programming language that runs inside the browser and can change the HTML and CSS while the page is open, in response to clicks, typing, or timers — this is what makes a “static” page still feel interactive.

DOM (Document Object Model). The browser’s live, in-memory representation of the page’s HTML, which JavaScript can read and rewrite. When this demo’s JavaScript does something like `document.getElementById("library-list").innerHTML = ...`, it is finding a specific piece of the page (identified by an `id` attribute in the HTML) and replacing everything inside it with new HTML text it builds on the fly.

SaaS (Software as a Service). A product delivered as a hosted web application that customers pay a recurring subscription for, rather than software they install and own a copy of. The real `agency-blog-saas` is a SaaS: agency clients log in, spend “credits” to generate blog content, and pay monthly or annual subscription tiers for a credit allowance.

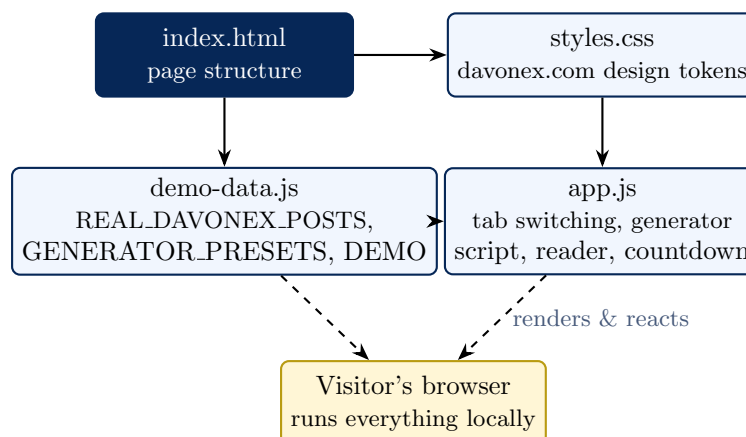
AEO / SEO. SEO (Search Engine Optimization) is writing and structuring web content so that classic search engines like Google rank it well. AEO (Answer/Agentic Engine Optimization) is the newer, related discipline of structuring content so that AI systems like ChatGPT or Google’s AI Overviews choose to cite or recommend it directly when answering a user’s question. The real blog posts reproduced in this demo are all AEO/SEO educational content the real pipeline actually published for a real e-commerce client.

3 The one honesty rule this whole repository is built around

Every other design decision in this project follows from a single constraint the README states explicitly: *no live network calls, ever, and everything fabricated must be visibly labeled as fabricated*. Concretely:

- There is no backend. No request ever leaves the browser once the page has loaded — not to the real `agency-blog-saas` product, not to `davonex.com`, not anywhere.
- Anything shown that is *real* (the 15 blog posts, their images, their full text, the subscription price tiers) was fetched once, ahead of time, by the developer, and is now baked directly into `demo-data.js` and `assets/posts/` as a frozen snapshot.
- Anything *fabricated* (the credit balance, the “Aurora Fitness Co.” client, the generator’s scripted timing) carries a visible label — a yellow banner, a “Demo” ribbon, explanatory copy — at the point where a visitor would see it, so real and fake content are never ambiguous side by side.

4 Architecture: how the four files fit together



`index.html` loads `styles.css` for appearance, then loads `demo-data.js` *before* `app.js` — this order matters, because `app.js` refers to the constants `demo-data.js` defines (`REAL_DAVONEX_POSTS`, `GENERATOR_PRESETS`, `DEMO`) as soon as the page finishes loading. There is no build step: these are the exact files the browser downloads and runs, unmodified.

5 File-by-file walkthrough

5.1 index.html — page structure

The document has, in order: a fixed yellow “Demo mode” disclosure banner; a sticky navigation bar; a hero section with the page title and a tab bar (four buttons: AI Blog Generator, Blog Library, Subscriptions, Client Updates); five `<section class="panel">` blocks, one per tab plus a fifth, hidden-by-default `panel-reader` used for the in-page article view; and a footer restating the demo disclaimer and the real project’s actual published stack and test numbers (“Django 4.2, PostgreSQL 18, Docker, Gunicorn, and nginx”; “132/132 tests passing, 56% coverage”). Only one panel is visible at a time — CSS’s `.panel { display: none; }` rule, overridden by an `.is-active` class that `app.js` toggles, is what makes tab switching work without ever reloading the page.

5.2 demo-data.js — the data layer

This file defines four things, all as plain JavaScript constants (no server, no database — just values sitting in memory once the script runs):

1. `REAL_DAVONEX_POSTS` — an array of 15 objects, one per real blog post published on `davonex.com/blog`, an actual configured client of the real production pipeline this demo stands in for. Each object carries a `slug`, `title`, `excerpt`, `byline`, `date`, `read time`, a local image path under `assets/posts/`, the *full* article body as an HTML string (`body_html`), and a `references` array of the article’s real citations (title, URL, date). The comment at the top of the file is explicit that this content is copied verbatim from the real, published pages, fetched once as of 2026-07-07, and never re-fetched.
2. `GENERATOR_PRESETS` — derived, not hand-written, by mapping over `REAL_DAVONEX_POSTS`: `const GENERATOR_PRESETS = REAL_DAVONEX_POSTS.map(p => {...})`. Each preset has the same `slug` as its source post plus a `topic`, `description`, and a `keywords` string computed from the title (lower-cased, punctuation stripped, split into words longer than three characters, first six joined with commas). Because this is a *derivation*, not an independent hand-authored list, every preset is guaranteed to correspond to exactly one real post.
3. `DEMO` — a small object holding the fabricated starting credit balance (1200), the three real subscription tiers (Basic \$9.99/mo for 500 credits, Pro \$24.99/mo for 1,500, Enterprise \$79.99/mo for 5,000 — the README states these are the real, already-published, non-client-specific tier prices), the one invented illustrative client ("*Aurora Fitness Co. (invented, illustrative client)*") with three fake update entries, and a `facts` object repeating the real test/coverage numbers also shown in the footer.

Every article body embeds numbered citation markers (`<sup class="citation">`) that link to entries in that post’s own `references` list — this is the real site’s own inline-citation style, copied along with the article text, not something this demo invented.

5.3 app.js — behavior

This file has no imports and makes no network calls (its own top-of-file comment says so explicitly). Its responsibilities, function by function:

- `escapeHtml(str)` — converts `&`, `<`, `>` in a string to their HTML-safe entity forms before that string is inserted into the page. This matters because several strings here (post titles, user-typed

form values) are later spliced directly into HTML with template literals; without escaping, a title containing a literal `<` character could break the page’s markup. This is a lightweight, hand-rolled version of what web frameworks normally do automatically.

- `renderCreditsBanner()` — writes the current `creditsBalance` variable into every element with class `.credits-value` on the page.
- `renderPackages()` — builds the three pricing cards from `DEMO.packages`.
- `autofillRandom()` — picks one of the 15 `GENERATOR.PRESETS` at random (`Math.floor(Math.random() * GENERATOR.PRESETS.length)`), remembers its slug in the module-level variable `currentPresetSlug`, and fills the Topic/Description/Keywords form fields with that preset’s values.
- `runGenerator(e)` — the scripted “AI generation.” It calls `e.preventDefault()` so the HTML form never actually submits anywhere, disables the Generate button, and then chains three `setTimeout` calls at 700ms, 1900ms, and 2900ms that rewrite a status line through “Checking credit balance...”, “Generating post...”, “Formatting and saving...”, and finally call `addToLibrary(post.slug)`. The post used is whichever real post `currentPresetSlug` points to (or the first preset if the visitor never clicked Autofill) — so the outcome is deterministic per autofill, not a fresh random pick on submit.
- `addToLibrary(slug)` — records a `libraryDemo` object with an `addedAt/expiresAt` pair 15 minutes apart, a `setInterval` that re-renders the library every second (to update the countdown text), and a `setTimeout` that clears the demo entry entirely after 15 minutes. If a visitor generates twice, the previous timers are cleared first so only one demo entry ever exists at a time.
- `renderLibrary()` — rebuilds the library grid: the fabricated demo card (if any, via `demoPostCard()`) first, then all 15 real posts (via `realPostCard(post)`), and updates the tab’s post count.
- `openReader(slug, isDemo)` — builds the full in-page article view: hero image, byline/date/read-time row, the post’s full `body_html`, a References list built from that post’s `references` array, a demo flag banner if this is the fabricated duplicate, and — for every post, real or duplicated — a provenance note linking back to the real `davonex.com/blog/{slug}/` URL.
- `switchTab(tabId)` — the whole navigation model: toggle `.is-active` on the matching tab button and the matching `#panel-{tabId}` section; every other panel simply has no class and stays hidden via the CSS rule from §5.1.
- The `DOMContentLoaded` listener at the bottom wires up every button and does the first render of credits, packages, library, and client updates when the page finishes loading.

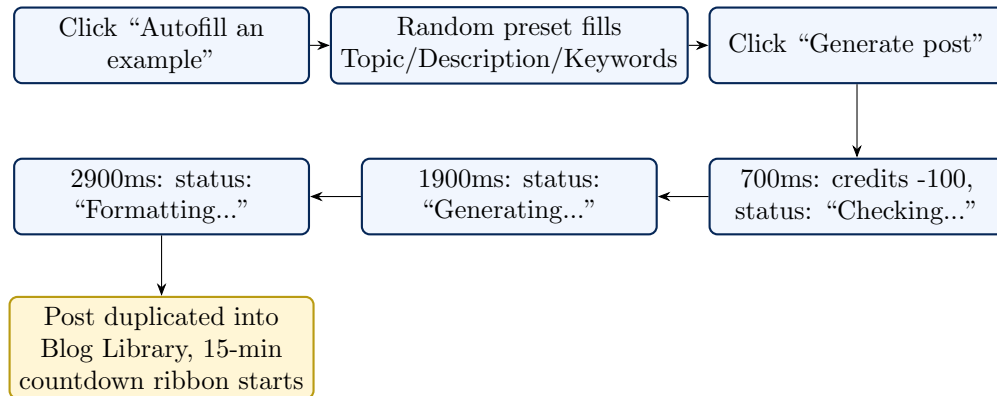
5.4 styles.css — the davonex.com visual language

The stylesheet begins with an explicit provenance comment: its color, radius, and shadow custom properties (`--blue: #136BEE`, `--navy: #062757`, `--r-lg: 26px`, and so on) were read directly off `davonex.com`’s own live CSS, per the owner’s explicit direction to style this specific demo after that site’s public design rather than `digitalise.agency`’s own indigo palette. The comment is careful to note this is *public visual design language* — what every visitor’s browser already renders when it loads that site — not any private server code, authentication logic, or billing internals, none of which are part of this repository. Component patterns (the pill-shaped `.btn`, the top-accent hover card, the sticky blurred nav, the article `.reader_prose` typography) are likewise copied 1:1 from `davonex.com`’s live CSS and one of its real rendered article pages.

Design tokens. Named CSS custom properties (`--blue`, `--r-lg`, etc., declared once under `:root`) that stand in for raw values throughout a stylesheet. Using tokens means changing one line updates every component that references it, and — as here — makes it easy to state precisely which literal values were copied from where.

6 User flows

6.1 The AI Blog Generator flow

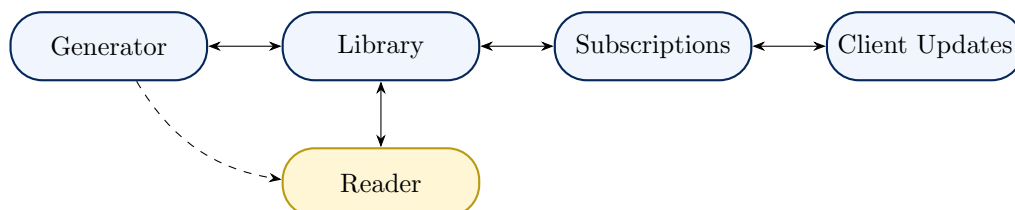


Because the post used at the end is fixed by whichever preset the autofill picked (step b), the same autofill always produces the same final post — the README calls this out directly as a deliberate design choice: “random fabrication on every click would undercut the ‘same input, same output’ premise a real generator implies.”

6.2 Reading an article

Clicking any card — real or the one fabricated duplicate — calls `openReader(slug, isDemo)`, which looks the post up in `REAL_DAVONEX_POSTS` by slug (the duplicate card points at the same real post’s slug, just rendered with a different card wrapper) and switches the page to the hidden `panel-reader` section. The “Back to Blog Library” button returns to whichever tab was active before the reader opened (`readerReturnTab`), so opening a post from the Generator’s status message returns there too, not always to the Library tab.

6.3 Tab state machine



Exactly one of the five panels is visible at any time; the reader is reachable from any tab that shows a post card and always remembers where to send the visitor back.

7 Data model detail

Field (per post)	Meaning
<code>slug</code>	Unique identifier, matches the real post’s URL path segment.
<code>title, excerpt</code>	Real headline and summary text, copied verbatim.
<code>by, date, read_time</code>	Real byline/date/read-time strings.
<code>image</code>	Local path under <code>assets/posts/</code> to a downloaded, re-sized copy of the real hero image.
<code>body_html</code>	The full real article body as an HTML string, including inline citation markers and any embedded tables.
<code>references</code>	Array of <code>{title, url, date}</code> — the article’s real numbered citation list.

8 How this relates to the real, private project

Screen	Real agency-blog-saas	This demo
AI Blog Generator	Calls a real Django backend, spends real credits, invokes a real LLM pipeline	Scripted timeouts, deducts a fabricated credit counter, always resolves to one of 15 pre-baked real posts
Blog Library	Per-client library backed by a real database	15 real, already-public posts for one real client, stored as a static JS array
Subscriptions	Real Stripe-backed billing	Static display of the real, published tier prices; “Redeem” validates nothing
Client Updates	Real per-client activity feed	One invented, fictional client (“Aurora Fitness Co.”)

9 Challenges the owner records in the README

- Making the generator’s outcome deterministic without a backend — solved by deriving presets directly from the real posts and tracking the last-autofilled slug, rather than generating something new each click.
- Sourcing real article content without breaking the “no live calls” rule — solved by fetching everything once and storing it locally in `assets/posts/` and `demo-data.js`, rather than linking out or hotlinking images from the real site on every page load.
- Deciding which real facts were safe to reuse — the subscription tiers and the README’s published test/coverage numbers are treated as safe (already public); anything client-specific beyond the one already-public `davonex.com` content library is excluded entirely.

10 Setup, deployment, and licensing

Setup: none. Open `index.html` in any browser, or serve the folder with any static file server.

Deployment: `netlify.toml` sets `[build] publish = "."`, meaning Netlify serves the repository root as-is, with no build command — consistent with there being no build step at all. **License:** the repository ships a PolyForm Strict License 1.0.0, copyright Salman Adnan, 2026 — a source-available license that permits viewing and noncommercial/personal use but not redistribution or commercial reuse. `assets/posts/CREDITS.md` separately documents that the 15 hero images and article bodies are the owner’s own agency’s real client work (not third-party stock under a different license), already disclosed elsewhere in the portfolio.

11 What the owner says they’d change

The README’s own “What I’d do differently” section notes that a real client publishes more content every month, so this static 15-post snapshot will drift out of date over time; a live version could re-sync periodically, but that was deliberately skipped here to keep the “no live calls” guarantee airtight.